

□ Exercice 1 : Conversions

Recopier et compléter :

Décimal	Binaire	Hexadécimal
$\overline{199}^{10}$	...	...
...	$\overline{100110101}^2$	...
...	...	$\overline{7AF}^{16}$
...	$\overline{11100010110}^2$	...
$\overline{2023}^{10}$	...	...
...	...	$\overline{1B2C}^{16}$

□ Exercice 2 : Dépassement de capacité

On suppose que $n = 100$ est un entier non signé représenté sur 8 bits en C (type `uint8_t`).

1. Quelles sont la plus grande et la plus petite valeur possibles de n ?
2. Après l'instruction $n = n + 50$, quelle sera la valeur de n ? Justifier.
3. On suppose que n vaut de nouveau 100, quelle sera sa valeur après l'instruction $n = n - 200$?
4. On suppose que n vaut de nouveau 100, quelle sera sa valeur après l'instruction $n = n - 150$?

□ Exercice 3 : Bug ! (d'après un exercice proposé par A. Domenech)

Dans un jeu vidéo, les points de vie d'un boss sont représentés par un `uint8_t`. Le boss démarre avec 199 points de vie. À chaque tour, le joueur inflige des dégâts au boss puis ce dernier se régénère de 60 points de vie. Cette régénération est codée de la façon suivante :

```
pv_boss = min(pv_boss+60,199);
```

1. Donner les points de vie du boss, si le joueur lui inflige 100 points de dégâts.
2. Donner les points de vie du boss, si le joueur lui inflige 0 point de dégât.
3. Montrer qu'il est facile de tuer le boss dès le premier tour de jeu, en lui infligeant pourtant moins de dégâts qu'il n'a de points de vie.
4. Proposer une modification de la régénération du boss afin de corriger ce bug.

□ Exercice 4 : Méthode pratique pour le complément à deux

Le but de l'exercice est de justifier la méthode vue en cours pour calculer la représentation en complément à deux d'un entier n sur p bits.

1. Rappeler cette méthode.
2. Étant donné un entier $-2^{p-1} < n < 0$, on note $b_{p-1} \dots b_0$ la suite de bits de la représentation binaire de $-n$, c'est-à-dire qu'on a :

$$-n = \sum_{k=0}^{p-1} b_k 2^k \text{ avec } b_{p-1} = 0$$

$$\text{Montrer que } n = -2^{p-1} + \sum_{k=0}^{p-2} b'_k 2^k + 1 \text{ où } b'_k = 1 - b_k \text{ pour } k \in \llbracket 0; p-2 \rrbracket.$$

□ Exercice 5 : Représentation en complément à deux

1. Quel est le nombre représenté en complément à 2 sur 8 bits par 10001111 ?
2. Quel est le nombre représenté en complément à 2 sur 8 bits par 11001101 ?
3. Donner la représentation en complément à deux sur 8 bits de -121 .
4. Donner la représentation en complément à deux sur 8 bits de -77 .

□ Exercice 6 : Capacité maximale

1. En supposant qu'on code les entiers non signés sur 10 bits, quel sera le plus grand entier représentable ?
2. Si on code les entiers signés en complément à 2 sur 10 bits, donner le plus petit et le plus grand entier représentables ainsi que leur écriture binaire.

□ **Exercice 7** : *Addition en complément à deux*

1. Coder en binaire sur un octet en complément à deux $\overline{75}^{10}$.
2. Même question pour $\overline{33}^{10}$.
3. Faire l'addition binaire de ces deux nombres.
4. Convertir en décimal pour vérifier qu'on obtient bien 42.

□ **Exercice 8** : *D'une écriture à l'autre*

1. Donner l'écriture décimale de $\overline{11000,011}^2$
2. Donner l'écriture décimale de $\overline{0,11011011}^2$
3. Donner l'écriture dyadique de $\overline{33,40625}^{10}$
4. Donner l'écriture dyadique de $\overline{0,7}^{10}$

□ **Exercice 9** : *Représentation des flottants*

1. Donner la valeur décimale des nombres suivants codés sous le format simple précision de la norme IEEE-754 :
 - a) $\underbrace{1}_{\text{signe}} \underbrace{01111101}_{\text{exposant décalé}} \underbrace{011011000000000000000000}_{\text{mantisse}}$
 - b) 1 10001001 110000000000000000000000
2. Donner la représentation flottante en simple précision au format de la norme IEEE-754 des nombres suivants :
 - a) $-16,75$.
 - b) $-0,2$.

□ **Exercice 10** : *Convergence d'une suite*

On considère la suite :

$$\begin{cases} u_0 = e - 1 \\ u_{n+1} = (n+1)u_n - 1 \end{cases}$$

Le but de l'exercice est d'établir que cette suite converge vers 0.

On note

$$S_n = \sum_{k=0}^n \frac{1}{k!}$$

On pourra utiliser sans justification le résultat suivant (qui sera démontré en mathématiques) : pour tout $n \in \mathbb{N}^*$: $S_n \leq e \leq S_n + \frac{1}{n n!}$

1. Montrer que $e = \lim_{n \rightarrow +\infty} S_n$
2. Montrer que pour tout $n \in \mathbb{N}$, $u_n = n!(e - S_n)$
3. En déduire que $(u_n)_{n \in \mathbb{N}}$ converge et donner sa limite.
4. Calculer les premiers termes de cette suite à l'aide de votre calculatrice. Commenter.

□ **Exercice 11** : *Convergence « numérique » et convergence mathématique*

On considère la suite :

$$\begin{cases} u_1 = \frac{5}{4} \\ u_2 = \frac{7}{5} \\ u_{n+2} = 10 - \frac{23}{u_{n+1}} + \frac{14}{u_n u_{n+1}} \end{cases}$$

1. Montrer que le terme général de $(u_n)_{n \in \mathbb{N}^*}$ est $u_n = \frac{2^n + 3}{2^{n-1} + 3}$.
2. Déterminer la limite de $(u_n)_{n \in \mathbb{N}^*}$.
3. Calculer les premiers termes de cette suite à l'aide de votre calculatrice. Commenter.

□ **Exercice 12** : *Caractérisation des nombres dyadiques*

Un nombre réel $x \in [0, 1[$ est dit dyadique s'il admet une écriture binaire finie, c'est-à-dire qu'il existe un

entier $p \in \mathbb{N}^*$ et des bits $b_1, \dots, b_p \in \{0, 1\}$ tels que :

$$x = \sum_{k=1}^p b_k 2^{-k}$$

1. Montrer qu'un nombre rationnel $x = \frac{a}{b}$ sous forme irréductible ($a, b \in \mathbb{N}^*$, $\text{pgcd}(a, b) = 1$) est dyadique si et seulement si son dénominateur b est une puissance de 2 (c'est-à-dire qu'il existe $k \in \mathbb{N}$ tel que $b = 2^k$).

- Si x est dyadique, alors :

$$x = \sum_{k=1}^p b_k 2^{-k} = \frac{\sum_{k=1}^p b_k 2^{p-k}}{2^p} = \frac{N}{2^p} \quad \text{avec } N \in \mathbb{N}$$

La fraction irréductible $\frac{a}{b}$ s'obtient en simplifiant $\frac{N}{2^p}$ par $\text{pgcd}(N, 2^p)$. Comme les seuls diviseurs de 2^p sont des puissances de 2, le dénominateur b est nécessairement de la forme 2^k (avec $k \leq p$).

- Réciproquement, si $b = 2^k$, alors $x = \frac{a}{2^k} = a \times 2^{-k}$. En écrivant la décomposition binaire de l'entier $a : a = \sum_{j=0}^m c_j 2^j$, on obtient :

$$x = \sum_{j=0}^m c_j 2^{j-k}$$

Comme $x \in [0, 1[$, on a $a < 2^k$, donc $m < k$ et les exposants $j - k$ sont tous strictement négatifs. Cela constitue bien une écriture binaire finie, donc x est dyadique.

2. Parmi les nombres suivants, déterminer ceux qui possèdent une écriture dyadique finie :

$$x_1 = \frac{7}{16} \quad x_2 = \frac{3}{25} \quad x_3 = 0,1 = \frac{1}{10} \quad x_4 = \frac{13}{64}$$

- $x_1 = \frac{7}{16} = \frac{7}{2^4}$: le dénominateur est une puissance de 2, l'écriture dyadique est finie ($x_1 = \overline{0,0111}^2$).
- $x_2 = \frac{3}{25} = \frac{3}{5^2}$: le dénominateur admet 5 comme facteur premier, l'écriture est donc infinie.
- $x_3 = \frac{1}{10} = \frac{1}{2 \times 5}$: le dénominateur admet 5 comme facteur premier, l'écriture est infinie (périodique : $\overline{0,000110011\dots}^2$).
- $x_4 = \frac{13}{64} = \frac{13}{2^6}$: le dénominateur est une puissance de 2, l'écriture est finie ($x_4 = \overline{0,001101}^2$).

3. En déduire pourquoi aucun ordinateur travaillant en base 2 ne peut représenter exactement le nombre 0,1, même en augmentant la taille de la mantisse.

Puisque $0,1 = \frac{1}{10}$ n'est pas un rationnel dyadique, son écriture en base 2 nécessite une infinité de bits. Tout format flottant (qu'il soit codé sur 32 bits, 64 bits ou davantage) ne disposant que d'un nombre fini de bits pour stocker la mantisse, le nombre 0,1 subit obligatoirement un arrondi, ce qui introduit une erreur de représentation intrinsèque.

□ Exercice 13 : Codage ASCII et bit de parité

Le standard ASCII d'origine n'utilise que 7 bits par caractère (codes de 0 à 127). Dans les télécommunications, le 8^e bit (bit de poids fort) servait historiquement de *bit de parité* pour détecter d'éventuelles erreurs de transmission sur la ligne.

Dans un protocole à **parité paire**, on choisit le bit de poids fort de sorte que le nombre total de bits à 1 dans l'octet soit **pair**.

- Donner la suite d'octets en hexadécimal représentant la chaîne "MP2I" en ASCII standard sur 7 bits (avec le bit de poids fort à 0).

D'après les codes ASCII :

- 'M' a pour code décimal $77 = 4 \times 16 + 13 = \overline{4D}^{16}$.
- 'P' a pour code décimal $80 = 5 \times 16 + 0 = \overline{50}^{16}$.
- '2' a pour code décimal $50 = 3 \times 16 + 2 = \overline{32}^{16}$.
- 'I' a pour code décimal $73 = 4 \times 16 + 9 = \overline{49}^{16}$.

La suite d'octets en hexadécimal est donc : **4D 50 32 49**.

- Pour chacun des caractères suivants, donner son code ASCII sur 7 bits en binaire, puis déterminer la valeur de l'octet complet (sur 8 bits) avec bit de parité paire :
 - Le caractère 'A' (code décimal 65).
 - Le caractère 'C' (code décimal 67).

- $65 = 64 + 1 = \overline{1000001}^2$. Cette séquence de 7 bits contient 2 bits à 1, ce qui est déjà pair. Le bit de parité (de poids fort) est donc mis à 0. L'octet complet est $\overline{01000001}^2$ (0x41).
- $67 = 64 + 2 + 1 = \overline{1000011}^2$. Cette séquence contient 3 bits à 1 (nombre impair). Le bit de parité est donc mis à 1 pour obtenir un total pair (4 bits à 1). L'octet complet est $\overline{11000011}^2$ (0xC3).

- Un récepteur reçoit l'octet $\overline{11000101}^2$. Détecte-t-il une erreur de transmission ? Justifier.

L'octet reçu $\overline{11000101}^2$ contient 4 bits valant 1. Le nombre de bits à 1 étant pair, la règle de parité est vérifiée : le récepteur ne détecte pas d'erreur simple.

- Quelle est la limite de cette méthode de détection d'erreur ? Que se passe-t-il si deux bits sont altérés simultanément lors du transfert ?

Si deux bits (ou plus généralement un nombre pair de bits) sont simultanément modifiés par le bruit lors de la transmission, la parité du nombre de bits à 1 reste inchangée. L'erreur n'est donc pas détectée par le récepteur. Le bit de parité ne permet de détecter que les erreurs portant sur un nombre impair de bits (et en particulier les erreurs isolées d'un unique bit).

□ Exercice 14 : Annulation catastrophique

On s'intéresse au calcul de la fonction $f(x) = \sqrt{x+1} - \sqrt{x}$ pour de grandes valeurs de x . On suppose qu'un système calcule avec une précision limitée à **4 chiffres significatifs** en base 10 (c'est-à-dire que chaque résultat intermédiaire est arrondi à 4 chiffres significatifs). On souhaite évaluer $f(1000)$.

- Sachant que $\sqrt{1001} \approx 31,63858$ et $\sqrt{1000} \approx 31,62277$, donner les représentations arrondies à 4 chiffres significatifs de ces deux nombres.

Avec 4 chiffres significatifs :

$$\sqrt{1001} \approx 31,64 \quad \text{et} \quad \sqrt{1000} \approx 31,62$$

- Effectuer la soustraction de ces deux valeurs arrondies. Combien de chiffres significatifs le résultat obtenu comporte-t-il ? Que constatez-vous ?

On calcule la différence :

$$31,64 - 31,62 = 0,02 = 2 \times 10^{-2}$$

Le résultat ne comporte plus qu'un **seul chiffre significatif** ! Les trois autres ont été complètement perdus lors de la soustraction de deux grandeurs très proches (phénomène d'*annulation catastrophique*).

La valeur exacte étant $f(1000) \approx 0,0158074$, l'erreur relative commise est énorme :

$$\left| \frac{0,02 - 0,01581}{0,01581} \right| \approx 26,5 \%$$

3. En multipliant par la quantité conjuguée, donner une autre expression de $f(x)$.

En multipliant et en divisant par $\sqrt{x+1} + \sqrt{x}$:

$$f(x) = \frac{(\sqrt{x+1} - \sqrt{x})(\sqrt{x+1} + \sqrt{x})}{\sqrt{x+1} + \sqrt{x}} = \frac{(x+1) - x}{\sqrt{x+1} + \sqrt{x}} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$

4. Calculer à nouveau $f(1000)$ avec cette nouvelle formule en utilisant la même précision de 4 chiffres significatifs. Comparer avec la valeur exacte $f(1000) \approx 0,0158074$.

Avec la nouvelle formule :

- Dénominateur : $31,64 + 31,62 = 63,26$ (addition de deux nombres positifs, aucun chiffre significatif n'est perdu).

- Inverse : $\frac{1}{63,26} \approx 0,0158077$, qui s'arrondit à 4 chiffres significatifs en 0,01581.

Le résultat conserve bien ses 4 chiffres significatifs et l'erreur relative par rapport à la valeur exacte est inférieure à 0,02 %.